

# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_65bdc027-8a2\agent-a6868d7f411435f00.jsonl`
- SHA-256: `9a6d28727413886fb813d7d0ad6d9359371b2d0d4ffa528d3fc62b3c38356025`
- Source modified: `2026-07-06T22:51:27+00:00`
- Imported at: `2026-07-08T16:00:06+00:00`
- project: `wf\_65bdc027-8a2`
- session\_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

## Transcript

### 1. user (2026-07-06T22:48:37.059Z)

Reviewing GitHub PR #53 "feat(policy): add F4 URL reputation stage" (branch feat/f4-url-reputation -> main). ADR 0012 F4.  
+357/-3, 3 files. Checked-out copy at C:/Users/avidu/Projects/\_wt-pr53 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f4-url-reputation:<path>').  
Run repros: PYTHONPATH=C:/Users/avidu/Projects/\_wt-pr53/src  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. (The 'regex' runtime dep is already installed in the venv.)

WHAT F4 DOES (policy.py):

- UrlReputationStage (Tier 3) under security\_checks.blocked\_domains. extract\_caption\_urls(text) uses a stdlib re regex \_URL\_RE.finditer  
  over content\_item.link\_url + content\_item.text (NO network I/O). normalize\_domain(): urlsplit -> hostname -> strip []/trailing-dot/casefold  
  -> ip\_address() for IP literals -> host.encode('idna') -> strip leading 'www.' -> \_valid\_domain\_name (labels regex, len<=253, has dot).
- parse\_domain\_reputation\_config: normalizes each blocked\_domain; invalid ones go to invalid\_domains (skipped, valid stay active); dedups.
- \_domain\_matches\_blocked(domain, blocked) = domain==blocked or domain.endswith('.'+blocked) [subdomain match].
- Fail-open on invalid config (pass\_ + warning + invalid\_domain\_reputation\_config=True). Stage order now: topic -> url -> caption via evaluate\_pipeline.

VERIFIED by main reviewer: gate green (533 passed @ 94.87%); subdomain match is correct (notbad.example NOT blocked by bad.example);  
\_URL\_RE is ReDoS-safe (adversarial inputs up to 56K chars all <0.03s ? mandatory '.' separators + bounded {0,61} prevent catastrophic backtracking);  
\_URL\_RE is DEVELOPER-controlled/fixed (not user-supplied like F3 patterns). Feature is dormant (no command sets blocked\_domains).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.

Severity: high (real bug/regression/exploitable), medium (real evasion/latent-DoS/design), low (acceptable-v1-limitation/hygiene). Distinguish false-NEGATIVE (evasion of a blocklist ? lets bad through, matches ADR 'v1 metadata' framing) from false-POSITIVE (wrongly blocks a legit domain ? worse, breaks delivery).

DIMENSION: stage wiring, config parsing, and consistency with F2/F3.

Investigate:

- Stage order topic -> url -> caption via evaluate\_pipeline; short-circuit correct?
- \_evaluate\_policy\_stages returns (passed, results); callers use passed.  
  Does url-stage rejection correctly block delivery and short-circuit caption? (end-to-end test asserts it ? verify independently.)
- parse\_domain\_reputation\_config vs parse\_caption\_safety\_config: same 'security\_checks' object

holds blocked\_domains AND unsafe\_caption\_terms/patterns.

Do the two stages parse the SAME security\_checks independently without interfering? If security\_checks has an invalid unrelated key, does one stage's

fail-open affect the other? (they load \*\_config separately from the raw config ? confirm no cross-contamination.)

- Fail-open granularity: invalid blocked\_domains skipped, valid stay active (test asserts).

Confirm parse doesn't raise on a bad domain (uses normalize->None).

What if blocked\_domains is not a list / contains non-strings -> \_string\_tuple raises

ValueError -> whole stage fail-open (disables domain rep). Same posture Q

as F3 (fail-open on a SAFETY stage lets bad through) ? but per-entry invalid is handled; only a wholesale type error disables. Assess.

- Does F4 read content\_item.link\_url which is never populated today (like text pre-F2)? Harmless (extra empty scan) or a latent dependency?

- Metadata-only claim: confirm no network/fetch/DNS anywhere (ip\_address does NOT resolve; encode('idna') is local).

Run repros. Report findings.

## 2. assistant (2026-07-06T22:48:43.055Z)

I'll review this PR's diff. Let me start by reading the changed files.

## 3. user (2026-07-06T22:48:44.176Z)

```
src/tg_video_filter/policy.py          | 219 ++++++
src/tg_video_filter/telethon_client.py |   3 +-
tests/test_policy.py                   | 138 ++++++
3 files changed, 357 insertions(+), 3 deletions(-)
```

## 4. user (2026-07-06T22:48:44.857Z)

```
diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
```

```
index e65789c..a90cf50 100644
```

```
--- a/src/tg_video_filter/policy.py
```

```
+++ b/src/tg_video_filter/policy.py
```

```
@@ -9,8 +9,11 @@ from __future__ import annotations
```

```
import json
import logging
+import re
from dataclasses import dataclass, field
+from ipaddress import ip_address
from typing import Protocol
+from urllib.parse import urlsplit
```

```
import regex
```

```
@@ -22,6 +25,18 @@ _MAX_CAPTION_SCAN_CHARS = 4096
```

```
_MAX_CAPTION_TERMS = 100
```

```
_MAX_CAPTION_PATTERNS = 25
```

```
_CAPTION_REGEX_TIMEOUT_SECONDS = 0.01
```

```
+_URL_RE = re.compile(
```

```
+    r"""(?ix)
```

```
+    (?<![@\\w.-])
```

```
+    (
```

```
+        (?:https?://|www\\.)[^\\s<>'"]+
```

```
+        |
```

```
+        (?:[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\\.)+[a-z]{2,}
```

```
+        (?:/[\\s<>'"]*)?
```

```
+    )
```

```
+    """
```

```
+)
```

```
+_TRAILING_URL_PUNCTUATION = ".,!?;:"
```

```

    @dataclass(frozen=True)
    @@ -103,6 +118,26 @@ class CaptionSafetyConfig:
        return bool(self.invalid_patterns or self.ignored_term_count or
self.ignored_pattern_count)

+@dataclass(frozen=True)
+class DomainReputationConfig:
+    """Parsed ADR 0012 URL/domain reputation config."""
+
+    blocked_domains: tuple[str, ...] = ()
+    invalid_domains: tuple[str, ...] = ()
+
+    @property
+    def enabled(self) -> bool:
+        return bool(self.blocked_domains)
+
+@dataclass(frozen=True)
+class ExtractedUrl:
+    """URL discovered from captured text with its normalized host domain."""
+
+    original_url: str
+    domain: str
+
+    def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
+        """Parse ``topics`` from a raw policy config object."""
+        raw_topics = config.get("topics")
    @@ -165,6 +200,35 @@ def parse_caption_safety_config(config: dict[str, object]) ->
CaptionSafetyConfi
    )

+def parse_domain_reputation_config(config: dict[str, object]) -> DomainReputationConfig:
+    """Parse URL/domain reputation settings from ``security_checks``."""
+    raw_security = config.get("security_checks")
+    if raw_security is None:
+        return DomainReputationConfig()
+    if not isinstance(raw_security, dict):
+        raise ValueError("security_checks must be a JSON object")
+
+    raw_domains = _string_tuple(
+        raw_security.get("blocked_domains"),
+        field_name="security_checks.blocked_domains",
+    )
+    blocked_domains: list[str] = []
+    invalid_domains: list[str] = []
+    seen: set[str] = set()
+    for raw_domain in raw_domains:
+        domain = normalize_domain(raw_domain)
+        if domain is None:
+            invalid_domains.append(raw_domain)
+            continue
+        if domain not in seen:
+            blocked_domains.append(domain)
+            seen.add(domain)
+    return DomainReputationConfig(
+        blocked_domains=tuple(blocked_domains),
+        invalid_domains=tuple(invalid_domains),
+    )
+
+def load_topic_preferences(policy: Policy) -> TopicPreferences:
+    """Load topic preferences from ``policy.config_json``."""

```

```

        decoded = json.loads(policy.config_json)
@@ -181,6 +245,14 @@ def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
    return parse_caption_safety_config(decoded)

+def load_domain_reputation_config(policy: Policy) -> DomainReputationConfig:
+    """Load URL/domain reputation settings from `policy.config_json`."""
+    decoded = json.loads(policy.config_json)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    return parse_domain_reputation_config(decoded)
+
+
def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
    if value is None:
        return ()
@@ -264,6 +336,146 @@ def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str,
...]:
    return tuple(topic for topic in topics if topic.casefold() in haystack)

+def normalize_domain(value: str) -> str | None:
+    """Normalize a configured or extracted domain for blocklist matching."""
+    text = value.strip()
+    if not text:
+        return None
+
+    parsed = urlsplit(text if "://" in text else f"//{text}")
+    host = parsed.hostname
+    if host is None:
+        return None
+
+    host = host.strip().strip("[]").rstrip(".").casefold()
+    if not host:
+        return None
+
+    try:
+        return ip_address(host).compressed.casefold()
+    except ValueError:
+        pass
+
+    try:
+        domain = host.encode("idna").decode("ascii").casefold()
+    except UnicodeError:
+        return None
+
+    if domain.startswith("www."):
+        domain = domain[4:]
+    if not _valid_domain_name(domain):
+        return None
+    return domain
+
+
+def extract_caption_urls(text: str | None) -> tuple[ExtractedUrl, ...]:
+    """Extract plain URLs from captured caption text without network I/O."""
+    if not text:
+        return ()
+
+    urls: list[ExtractedUrl] = []
+    for match in _URL_RE.finditer(text):
+        raw_url = match.group(1).rstrip(_TRAILING_URL_PUNCTUATION)
+        while raw_url.endswith("(") and raw_url.count("(") < raw_url.count(")"):
+            raw_url = raw_url[:-1]
+        domain = normalize_domain(raw_url)
+        if domain is not None:
+            urls.append(ExtractedUrl(original_url=raw_url, domain=domain))

```

```

+     return tuple(urls)
+
+
+def _valid_domain_name(domain: str) -> bool:
+    if len(domain) > 253 or "." not in domain:
+        return False
+    labels = domain.split(".")
+    return all(
+        re.fullmatch(r"[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?", label) is not None for label in
labels
+    )
+
+
+def _domain_matches_blocked(domain: str, blocked_domain: str) -> bool:
+    return domain == blocked_domain or domain.endswith(f".{blocked_domain}")
+
+
+class UrlReputationStage:
+    """ADR 0012 Tier 3 captured-URL domain blocklist checks."""
+
+    stage_name = "url-reputation"
+
+    async def evaluate(self, ctx: PolicyContext) -> StageResult:
+        try:
+            config = load_domain_reputation_config(ctx.policy)
+        except (json.JSONDecodeError, ValueError) as exc:
+            log.warning(
+                "domain reputation disabled due to invalid config policy_id=%s",
+                ctx.policy.id,
+                exc_info=True,
+            )
+            return StageResult.pass_(
+                self.stage_name,
+                f"invalid domain reputation config ignored: {exc}",
+                invalid_domain_reputation_config=True,
+            )
+
+        if config.invalid_domains:
+            log.warning(
+                "domain reputation ignored invalid domains policy_id=%s invalid_domains=%s",
+                ctx.policy.id,
+                list(config.invalid_domains),
+            )
+
+        if not config.enabled:
+            metadata = _url_reputation_metadata((), config)
+            if config.invalid_domains:
+                return StageResult.pass_(
+                    self.stage_name,
+                    "domain reputation has no valid blocked domains",
+                    **metadata,
+                )
+            return StageResult.pass_(self.stage_name, "domain reputation disabled")
+
+        urls = extract_captions_urls(ctx.content_item.link_url) + extract_captions_urls(
+            ctx.content_item.text
+        )
+
+        if not urls:
+            return StageResult.pass_(
+                self.stage_name,
+                "no links to scan",
+                **_url_reputation_metadata(urls, config),
+            )
+
+        for url in urls:
+            for blocked_domain in config.blocked_domains:

```

```

+         if _domain_matches_blocked(url.domain, blocked_domain):
+             return StageResult.fail(
+                 self.stage_name,
+                 f"blocked domain matched: {blocked_domain}",
+                 **_url_reputation_metadata(urls, config),
+                 blocked_domain=url.domain,
+                 matched_blocklist_domain=blocked_domain,
+                 matched_url=url.original_url,
+             )
+
+     return StageResult.pass_(
+         self.stage_name,
+         "links passed domain reputation checks",
+         **_url_reputation_metadata(urls, config),
+     )
+
+
+def _url_reputation_metadata(
+    urls: tuple[ExtractedUrl, ...],
+    config: DomainReputationConfig,
+) -> dict[str, object]:
+    domains = sorted({url.domain for url in urls})
+    metadata: dict[str, object] = {
+        "link_count": len(urls),
+        "domains": domains,
+        "blocked_domain_count": len(config.blocked_domains),
+    }
+    if config.invalid_domains:
+        metadata["ignored_invalid_domains"] = list(config.invalid_domains)
+    return metadata
+
+
+class CaptionSafetyStage:
+    """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
+
+@@ -420,14 +632,21 @@ async def evaluate_pipeline(
+    __all__ = [
+        "CaptionSafetyConfig",
+        "CaptionSafetyStage",
+    +    "DomainReputationConfig",
+    +    "ExtractedUrl",
+        "PolicyContext",
+        "PolicyStage",
+        "StageResult",
+        "TopicPreferenceStage",
+        "TopicPreferences",
+    +    "UrlReputationStage",
+        "evaluate_pipeline",
+    +    "extract_caption_urls",
+        "load_caption_safety_config",
+    +    "load_domain_reputation_config",
+        "load_topic_preferences",
+    +    "normalize_domain",
+        "parse_caption_safety_config",
+    +    "parse_domain_reputation_config",
+        "parse_topic_preferences",
+    ]
+diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
+index 011669b..3c7d534 100644
+--- a/src/tg_video_filter/telethon_client.py
++++ b/src/tg_video_filter/telethon_client.py
+@@ -49,6 +49,7 @@ from tg_video_filter.policy import (
+     PolicyContext,
+     StageResult,
+     TopicPreferenceStage,

```

```

+     UrlReputationStage,
+     evaluate_pipeline,
+ )

@@ -438,7 +439,7 @@ async def _evaluate_policy_stages(
    return True, []
    return await evaluate_pipeline(
        PolicyContext(content_item=content_item, source=source, policy=policy),
-        [TopicPreferenceStage(), CaptionSafetyStage()],
+        [TopicPreferenceStage(), UrlReputationStage(), CaptionSafetyStage()],
    )

diff --git a/tests/test_policy.py b/tests/test_policy.py
index 042c37b..e792ef7 100644
--- a/tests/test_policy.py
+++ b/tests/test_policy.py
@@ -9,12 +9,18 @@ from tg_video_filter.models import ContentItem, Policy, Source
from tg_video_filter.policy import (
    CaptionSafetyConfig,
    CaptionSafetyStage,
+    DomainReputationConfig,
+    ExtractedUrl,
    PolicyContext,
    StageResult,
    TopicPreferences,
    TopicPreferenceStage,
+    UrlReputationStage,
+    evaluate_pipeline,
+    extract_caption_urls,
+    normalize_domain,
+    parse_caption_safety_config,
+    parse_domain_reputation_config,
+    parse_topic_preferences,
)
from tg_video_filter.telethon_client import _evaluate_policy_stages
@@ -216,6 +222,115 @@ async def
test_topic_preferences_invalid_config_fails_open_with_explanation() ->
    assert result.metadata == {"invalid_topic_config": True}

+
+## --- UrlReputationStage -----
+
+
+def test_normalize_domain_strips_scheme_port_www_case_and_trailing_dot() -> None:
+    assert normalize_domain("HTTPS://WWW.Example.COM:443/path") == "example.com"
+    assert normalize_domain("sub.Example.com.") == "sub.example.com"
+    assert normalize_domain("https://[2001:db8::1]:443/path") == "2001:db8::1"
+    assert normalize_domain("not a domain") is None
+
+
+def test_extract_caption_urls_handles_plain_urls() -> None:
+    assert extract_caption_urls(
+        "See example.com/path, https://www.Bad.Example/a?b=1. and mail me@example.com"
+    ) == (
+        ExtractedUrl(original_url="example.com/path", domain="example.com"),
+        ExtractedUrl(original_url="https://www.Bad.Example/a?b=1", domain="bad.example"),
+    )
+
+
+def test_parse_domain_reputation_config_normalizes_and_records_invalid_domains() -> None:
+    assert parse_domain_reputation_config(
+        {"security_checks": {"blocked_domains": ["WWW.Bad.Example", "bad.example", "nope"]}}
+    ) == DomainReputationConfig(
+        blocked_domains=("bad.example",),

```

```

+         invalid_domains=("nope",),
+     )
+
+
+async def test_url_reputation_disabled_passes() -> None:
+    result = await UrlReputationStage().evaluate(_ctx(text="https://bad.example/a"))
+
+    assert result.passed is True
+    assert result.reason == "domain reputation disabled"
+
+
+async def test_url_reputation_blocked_domain_rejects_subdomain() -> None:
+    result = await UrlReputationStage().evaluate(
+        _ctx(
+            policy_json='{"security_checks":{"blocked_domains":["bad.example"]}}',
+            text="Watch https://cdn.bad.example/video now",
+        )
+    )
+
+    assert result.passed is False
+    assert result.reason == "blocked domain matched: bad.example"
+    assert result.metadata == {
+        "link_count": 1,
+        "domains": ["cdn.bad.example"],
+        "blocked_domain_count": 1,
+        "blocked_domain": "cdn.bad.example",
+        "matched_blocklist_domain": "bad.example",
+        "matched_url": "https://cdn.bad.example/video",
+    }
+
+
+async def test_url_reputation_allowed_domain_passes() -> None:
+    result = await UrlReputationStage().evaluate(
+        _ctx(
+            policy_json='{"security_checks":{"blocked_domains":["bad.example"]}}',
+            text="Watch https://good.example/video",
+        )
+    )
+
+    assert result.passed is True
+    assert result.reason == "links passed domain reputation checks"
+    assert result.metadata == {
+        "link_count": 1,
+        "domains": ["good.example"],
+        "blocked_domain_count": 1,
+    }
+
+
+async def test_url_reputation_no_link_caption_passes() -> None:
+    result = await UrlReputationStage().evaluate(
+        _ctx(
+            policy_json='{"security_checks":{"blocked_domains":["bad.example"]}}',
+            text="plain caption with no link",
+        )
+    )
+
+    assert result.passed is True
+    assert result.reason == "no links to scan"
+    assert result.metadata == {
+        "link_count": 0,
+        "domains": [],
+        "blocked_domain_count": 1,
+    }
+
+
+

```



```

+async def test_url_reputation_invalid_domain_keeps_valid_blocklist_active() -> None:
+    result = await UrlReputationStage().evaluate(
+        _ctx(
+            policy_json=('{"security_checks":{"blocked_domains":["bad.example","not a
domain"]}}'),
+            text="https://bad.example/path",
+        )
+    )
+
+    assert result.passed is False
+    assert result.metadata == {
+        "link_count": 1,
+        "domains": ["bad.example"],
+        "blocked_domain_count": 1,
+        "ignored_invalid_domains": ["not a domain"],
+        "blocked_domain": "bad.example",
+        "matched_blocklist_domain": "bad.example",
+        "matched_url": "https://bad.example/path",
+    }
+
+
+    # --- CaptionSafetyStage -----
+
@@ -383,7 +498,10 @@ async def test_evaluate_policy_stages_uses_default_policy_config() -> None:
    111,
    {
        "topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"},
        "security_checks": {"unsafe_caption_terms": ["malware"]},
+        "security_checks": {
+            "blocked_domains": ["bad.example"],
+            "unsafe_caption_terms": ["malware"],
+        },
    },
)
item = ContentItem(
@@ -398,10 +516,11 @@ async def test_evaluate_policy_stages_uses_default_policy_config() ->
None:

    assert [result.stage_name for result in results] == [
        "topic-preferences",
+        "url-reputation",
+        "caption-safety",
    ]
    assert policy_passed is True
-    assert [result.passed for result in results] == [True, True]
+    assert [result.passed for result in results] == [True, True, True]
    assert results[0].metadata == {"matched_include_topics": ["AI"]}

    topic_passed, topic_rejected = await _evaluate_policy_stages(
@@ -416,6 +535,20 @@ async def test_evaluate_policy_stages_uses_default_policy_config() -> None:
    assert topic_rejected[0].passed is False
    assert topic_rejected[0].metadata == {"matched_exclude_topics": ["Crypto"]}

+    url_passed, url_rejected = await _evaluate_policy_stages(
+        conn,
+        111,
+        src,
+        item.model_copy(update={"text": "AI https://bad.example/path"}),
+    )
+
+    assert url_passed is False
+    assert [result.stage_name for result in url_rejected] == [
+        "topic-preferences",
+        "url-reputation",

```

```

+         ]
+         assert url_rejected[-1].metadata["matched_blocklist_domain"] == "bad.example"
+
+         caption_passed, caption_rejected = await _evaluate_policy_stages(
+             conn,
+             111,
@@ -426,6 +559,7 @@ async def test_evaluate_policy_stages_uses_default_policy_config() -> None:
+         assert caption_passed is False
+         assert [result.stage_name for result in caption_rejected] == [
+             "topic-preferences",
+             "url-reputation",
+             "caption-safety",
+         ]
+         assert caption_rejected[-1].passed is False

```

## 5. assistant (2026-07-06T22:48:48.230Z)

Let me look at the surrounding code to verify wiring and consistency with F2/F3.

## 6. user (2026-07-06T22:48:49.258Z)

```

336         return tuple(topic for topic in topics if topic.casefold() in haystack)
337
338
339     def normalize_domain(value: str) -> str | None:
340         """Normalize a configured or extracted domain for blocklist matching."""
341         text = value.strip()
342         if not text:
343             return None
344
345         parsed = urlsplit(text if "://" in text else f"//{text}")
346         host = parsed.hostname
347         if host is None:
348             return None
349
350         host = host.strip().strip("[]").rstrip(".").casefold()
351         if not host:
352             return None
353
354         try:
355             return ip_address(host).compressed.casefold()
356         except ValueError:
357             pass
358
359         try:
360             domain = host.encode("idna").decode("ascii").casefold()
361         except UnicodeError:
362             return None
363         if domain.startswith("www."):
364             domain = domain[4:]
365         if not _valid_domain_name(domain):
366             return None
367         return domain
368
369
370     def extract_caption_urls(text: str | None) -> tuple[ExtractedUrl, ...]:
371         """Extract plain URLs from captured caption text without network I/O."""
372         if not text:
373             return ()
374
375         urls: list[ExtractedUrl] = []
376         for match in _URL_RE.finditer(text):
377             raw_url = match.group(1).rstrip(_TRAILING_URL_PUNCTUATION)
378             while raw_url.endswith("(") and raw_url.count("(") < raw_url.count(")"):
379                 raw_url = raw_url[:-1]

```

```

380         domain = normalize_domain(raw_url)
381         if domain is not None:
382             urls.append(ExtractedUrl(original_url=raw_url, domain=domain))
383     return tuple(urls)
384
385
386     def _valid_domain_name(domain: str) -> bool:
387         if len(domain) > 253 or "." not in domain:
388             return False
389         labels = domain.split(".")
390         return all(
391             re.fullmatch(r"[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?", label) is not None for
label in labels
392         )
393
394
395     def _domain_matches_blocked(domain: str, blocked_domain: str) -> bool:
396         return domain == blocked_domain or domain.endswith(f".{blocked_domain}")
397
398
399     class UrlReputationStage:
400         """ADR 0012 Tier 3 captured-URL domain blocklist checks."""
401
402         stage_name = "url-reputation"
403
404         async def evaluate(self, ctx: PolicyContext) -> StageResult:
405             try:
406                 config = load_domain_reputation_config(ctx.policy)
407             except (json.JSONDecodeError, ValueError) as exc:
408                 log.warning(
409                     "domain reputation disabled due to invalid config policy_id=%s",
410                     ctx.policy.id,
411                     exc_info=True,
412                 )
413                 return StageResult.pass_(
414                     self.stage_name,
415                     f"invalid domain reputation config ignored: {exc}",
416                     invalid_domain_reputation_config=True,
417                 )
418
419                 if config.invalid_domains:
420                     log.warning(
421                         "domain reputation ignored invalid domains policy_id=%s
invalid_domains=%s",
422                         ctx.policy.id,
423                         list(config.invalid_domains),
424                     )
425                 if not config.enabled:
426                     metadata = _url_reputation_metadata((), config)
427                     if config.invalid_domains:
428                         return StageResult.pass_(
429                             self.stage_name,
430                             "domain reputation has no valid blocked domains",
431                             **metadata,
432                         )
433                 return StageResult.pass_(self.stage_name, "domain reputation disabled")
434
435             urls = extract_captions_urls(ctx.content_item.link_url) + extract_captions_urls(
436                 ctx.content_item.text
437             )
438             if not urls:
439                 return StageResult.pass_(
440                     self.stage_name,
441                     "no links to scan",
442                     **_url_reputation_metadata(urls, config),

```

```

443         )
444
445     for url in urls:
446         for blocked_domain in config.blocked_domains:
447             if _domain_matches_blocked(url.domain, blocked_domain):
448                 return StageResult.fail(
449                     self.stage_name,
450                     f"blocked domain matched: {blocked_domain}",
451                     **_url_reputation_metadata(urls, config),
452                     blocked_domain=url.domain,
453                     matched_blocklist_domain=blocked_domain,
454                     matched_url=url.original_url,
455                 )
456
457     return StageResult.pass_(
458         self.stage_name,
459         "links passed domain reputation checks",
460         **_url_reputation_metadata(urls, config),
461     )
462
463
464 def _url_reputation_metadata(
465     urls: tuple[ExtractedUrl, ...],
466     config: DomainReputationConfig,
467 ) -> dict[str, object]:
468     domains = sorted({url.domain for url in urls})
469     metadata: dict[str, object] = {
470         "link_count": len(urls),
471         "domains": domains,
472         "blocked_domain_count": len(config.blocked_domains),
473     }
474     if config.invalid_domains:
475         metadata["ignored_invalid_domains"] = list(config.invalid_domains)
476     return metadata
477
478
479 class CaptionSafetyStage:
480     """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
481
482     stage_name = "caption-safety"
483
484     async def evaluate(self, ctx: PolicyContext) -> StageResult:
485         try:

```

## 7. user (2026-07-06T22:48:50.182Z)

```

480     """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
481
482     stage_name = "caption-safety"
483
484     async def evaluate(self, ctx: PolicyContext) -> StageResult:
485         try:
486             config = load_caption_safety_config(ctx.policy)
487             except (json.JSONDecodeError, ValueError) as exc:
488                 log.warning(
489                     "caption safety disabled due to invalid config policy_id=%s",
490                     ctx.policy.id,
491                     exc_info=True,
492                 )
493             return StageResult.pass_(
494                 self.stage_name,
495                 f"invalid caption safety config ignored: {exc}",
496                 invalid_caption_safety_config=True,
497             )
498         if not config.enabled:

```

```

499         metadata = _caption_safety_metadata(config, caption_present=False)
500     if config.has_config_warnings:
501         log.warning(
502             "caption safety has no valid active checks policy_id=%s
metadata=%s",
503             ctx.policy.id,
504             metadata,
505         )
506     return StageResult.pass_(
507         self.stage_name,
508         "caption safety has no valid active checks",
509         **metadata,
510     )
511     return StageResult.pass_(self.stage_name, "caption safety disabled")
512
513 if config.has_config_warnings:
514     log.warning(
515         "caption safety ignored invalid or excess config policy_id=%s "
516         "invalid_patterns=%s ignored_terms=%s ignored_patterns=%s",
517         ctx.policy.id,
518         list(config.invalid_patterns),
519         config.ignored_term_count,
520         config.ignored_pattern_count,
521     )
522
523 caption = (ctx.content_item.text or "").strip()
524 if not caption:
525     return StageResult.pass_(
526         self.stage_name,
527         "no caption to scan",
528         **_caption_safety_metadata(config, caption_present=False),
529     )
530
531 scan_caption = caption[:_MAX_CAPTION_SCAN_CHARS]
532 caption_truncated = len(caption) > _MAX_CAPTION_SCAN_CHARS
533 haystack = scan_caption.casefold()
534 for term in config.unsafe_terms:
535     if term.casefold() in haystack:
536         return StageResult.fail(
537             self.stage_name,
538             f"unsafe caption term matched: {term}",
539             **_caption_safety_metadata(
540                 config,
541                 caption_present=True,
542                 caption_truncated=caption_truncated,
543             ),
544             match_type="term",
545             matched_caption_term=term,
546         )
547
548 timed_out_patterns: list[str] = []
549 for pattern in config.unsafe_patterns:
550     try:
551         matched_pattern = regex.search(
552             pattern,
553             scan_caption,
554             flags=regex.IGNORECASE,
555             timeout=_CAPTION_REGEX_TIMEOUT_SECONDS,
556         )
557     except TimeoutError:
558         timed_out_patterns.append(pattern)
559     log.warning(
560         "caption safety regex timed out policy_id=%s pattern=%r",
561         ctx.policy.id,
562         pattern,

```

```

563         )
564         continue
565
566     if matched_pattern:
567         return StageResult.fail(
568             self.stage_name,
569             "unsafe caption pattern matched",
570             **_caption_safety_metadata(
571                 config,
572                 caption_present=True,
573                 caption_truncated=caption_truncated,
574                 timed_out_patterns=tuple(timed_out_patterns),
575             ),
576             match_type="pattern",
577             matched_caption_pattern=pattern,
578         )
579
580     return StageResult.pass_(
581         self.stage_name,
582         "caption passed safety checks",
583         **_caption_safety_metadata(
584             config,
585             caption_present=True,
586             caption_truncated=caption_truncated,
587             timed_out_patterns=tuple(timed_out_patterns),
588         ),
589     )
590
591
592 def _caption_safety_metadata(
593     config: CaptionSafetyConfig,
594     *,
595     caption_present: bool,
596     caption_truncated: bool = False,
597     timed_out_patterns: tuple[str, ...] = (),
598 ) -> dict[str, object]:
599     metadata: dict[str, object] = {
600         "caption_present": caption_present,
601         "checked_terms": len(config.unsafe_terms),
602         "checked_patterns": len(config.unsafe_patterns),
603     }
604     if caption_truncated:
605         metadata["caption_truncated"] = True
606         metadata["max_caption_scan_chars"] = _MAX_CAPTION_SCAN_CHARS
607     if config.invalid_patterns:
608         metadata["ignored_invalid_caption_patterns"] = list(config.invalid_patterns)
609     if config.ignored_term_count:
610         metadata["ignored_caption_term_count"] = config.ignored_term_count
611     if config.ignored_pattern_count:
612         metadata["ignored_caption_pattern_count"] = config.ignored_pattern_count
613     if timed_out_patterns:
614         metadata["timed_out_caption_patterns"] = list(timed_out_patterns)
615     return metadata
616
617
618 async def evaluate_pipeline(
619     ctx: PolicyContext,
620     stages: list[PolicyStage],
621 ) -> tuple[bool, list[StageResult]]:
622     """Evaluate stages in order and short-circuit on the first failure."""
623     results: list[StageResult] = []
624     for stage in stages:
625         result = await stage.evaluate(ctx)
626         results.append(result)
627         if not result.passed:

```

```

628         return False, results
629     return True, results
630
631
632     __all__ = [
633         "CaptionSafetyConfig",
634         "CaptionSafetyStage",
635         "DomainReputationConfig",
636         "ExtractedUrl",
637         "PolicyContext",
638         "PolicyStage",
639         "StageResult",

```

## 8. assistant (2026-07-06T22:48:53.567Z)

Now let me look at the caption safety config parsing (for the F2/F3 cross-contamination question) and the `\_MAX\_CAPTION\_SCAN\_CHARS` handling ? note the caption stage truncates to 4096 chars before scanning, but the URL stage does NOT truncate. Let me check the caption safety parse and the `\_string\_tuple` limit handling.

## 9. user (2026-07-06T22:48:55.176Z)

```

168     raw_security = config.get("security_checks")
169     if raw_security is None:
170         return CaptionSafetyConfig()
171     if not isinstance(raw_security, dict):
172         raise ValueError("security_checks must be a JSON object")
173
174     raw_terms = _string_tuple(
175         raw_security.get("unsafe_caption_terms"),
176         field_name="security_checks.unsafe_caption_terms",
177     )
178     raw_patterns = _string_tuple(
179         raw_security.get("unsafe_caption_patterns"),
180         field_name="security_checks.unsafe_caption_patterns",
181     )
182
183     unsafe_terms = raw_terms[:_MAX_CAPTION_TERMS]
184     unsafe_patterns: list[str] = []
185     invalid_patterns: list[str] = []
186     for pattern in raw_patterns[:_MAX_CAPTION_PATTERNS]:
187         try:
188             regex.compile(pattern, regex.IGNORECASE)
189         except regex.error:
190             invalid_patterns.append(pattern)
191         else:
192             unsafe_patterns.append(pattern)
193
194     return CaptionSafetyConfig(
195         unsafe_terms=unsafe_terms,
196         unsafe_patterns=tuple(unsafe_patterns),
197         invalid_patterns=tuple(invalid_patterns),
198         ignored_term_count=max(0, len(raw_terms) - _MAX_CAPTION_TERMS),
199         ignored_pattern_count=max(0, len(raw_patterns) - _MAX_CAPTION_PATTERNS),
200     )
201
202
203 def parse_domain_reputation_config(config: dict[str, object]) -> DomainReputationConfig:
204     """Parse URL/domain reputation settings from ``security_checks``."""
205     raw_security = config.get("security_checks")
206     if raw_security is None:
207         return DomainReputationConfig()
208     if not isinstance(raw_security, dict):
209         raise ValueError("security_checks must be a JSON object")
210

```

```

211     raw_domains = _string_tuple(
212         raw_security.get("blocked_domains"),
213         field_name="security_checks.blocked_domains",
214     )
215     blocked_domains: list[str] = []
216     invalid_domains: list[str] = []
217     seen: set[str] = set()
218     for raw_domain in raw_domains:
219         domain = normalize_domain(raw_domain)
220         if domain is None:
221             invalid_domains.append(raw_domain)
222             continue
223         if domain not in seen:
224             blocked_domains.append(domain)
225             seen.add(domain)
226     return DomainReputationConfig(
227         blocked_domains=tuple(blocked_domains),
228         invalid_domains=tuple(invalid_domains),
229     )
230
231
232 def load_topic_preferences(policy: Policy) -> TopicPreferences:
233     """Load topic preferences from ``policy.config_json``."""
234     decoded = json.loads(policy.config_json)
235     if not isinstance(decoded, dict):
236         raise ValueError("policy config_json must decode to a JSON object")
237     return parse_topic_preferences(decoded)
238
239
240 def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
241     """Load caption safety settings from ``policy.config_json``."""
242     decoded = json.loads(policy.config_json)
243     if not isinstance(decoded, dict):
244         raise ValueError("policy config_json must decode to a JSON object")
245     return parse_caption_safety_config(decoded)
246
247
248 def load_domain_reputation_config(policy: Policy) -> DomainReputationConfig:
249     """Load URL/domain reputation settings from ``policy.config_json``."""
250     decoded = json.loads(policy.config_json)
251     if not isinstance(decoded, dict):
252         raise ValueError("policy config_json must decode to a JSON object")
253     return parse_domain_reputation_config(decoded)
254
255
256 def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
257     if value is None:
258         return ()
259     if not isinstance(value, list):
260         raise ValueError(f"{field_name} must be an array")
261     values: list[str] = []
262     for item in value:
263         if not isinstance(item, str):
264             raise ValueError(f"{field_name} must contain strings")
265         text = item.strip()
266         if text:
267             values.append(text)

```

## 10. assistant (2026-07-06T22:49:02.116Z)

Now I have a clear picture. Let me investigate several concerns with repros. Key observations:

1. Caption stage truncates to `\_MAX\_CAPTION\_SCAN\_CHARS` (4096) before scanning; URL stage has NO such cap ? it scans the full untruncated `text` AND `link\_url`. That's a divergence but "verified ReDoS-safe."



2. The `matched\_url` returned in metadata for a subdomain is `url.original\_url` but `blocked\_domain=url.domain` ? checking consistency.

3. Cross-contamination: both stages parse the same `security\_checks`. If `unsafe\_caption\_patterns` is invalid type (non-list), caption stage fails open but URL stage does NOT (it doesn't read that key). Conversely if `blocked\_domains` is non-list, URL stage fails open but caption stage stays active. Let me verify.

Let me check the `\_string\_tuple` non-list behavior for the cross-contamination and check the trailing-punctuation stripping edge cases.

#### 11. user (2026-07-06T22:49:12.320Z)

```
=== cross-contamination: invalid unsafe_caption_patterns type, valid blocked_domains ===
domain parse: DomainReputationConfig(blocked_domains=('bad.example',), invalid_domains=())
caption parse RAISED: ValueError security_checks.unsafe_caption_patterns must be an array
```

```
=== cross-contamination: invalid blocked_domains type, valid terms ===
domain parse RAISED: ValueError security_checks.blocked_domains must be an array
caption parse: CaptionSafetyConfig(unsafe_terms=('malware',), unsafe_patterns=(),
invalid_patterns=(), ignored_term_count=0, ignored_pattern_count=0)
```

```
=== non-string entry in blocked_domains -> whole stage fail-open ===
domain parse RAISED: ValueError security_checks.blocked_domains must contain strings
```

#### 12. assistant (2026-07-06T22:49:17.703Z)

Good ? no cross-contamination between the two stages; they parse independently. Now let me probe evasion vectors in URL extraction and normalization: userinfo `@`, port edge cases, IDN, and the trailing-punctuation/paren stripping.

#### 13. user (2026-07-06T22:49:30.557Z)

```
=== userinfo @ (lookbehind blocks leading @/word) ===
'https://bad.example@evil.example/path' -> [('https://bad.example@evil.example/path',
'evil.example')]
'Visit user:pw@bad.example/x' -> []
'text https://user@bad.example/x' -> [('https://user@bad.example/x', 'bad.example')]

=== trailing paren / punctuation ===
'(https://bad.example/path)' -> [('https://bad.example/path', 'bad.example')]
'see https://bad.example/wiki_(page)' -> [('https://bad.example/wiki_(page)', 'bad.example')]
'https://bad.example/a).' -> [('https://bad.example/a', 'bad.example')]

=== schemeless with path but leading @ or word boundary ===
'email foo@bad.example' -> []
'@bad.example' -> []
'x.bad.example' -> [('x.bad.example', 'x.bad.example')]

=== IDN / unicode homoglyph & punycode ===
normalize xn--: xn--80ak6aa92e.com
normalize unicode: xn--pple-43d.com
normalize idn ascii vs unicode match:
  config unicode: xn--mnchen-3ya.example
  url punycode: xn--mnchen-3ya.example

=== case where blocked config has trailing path/port ===
normalize 'bad.example:8080/x': bad.example
normalize 'http://bad.example': bad.example
```

#### 14. assistant (2026-07-06T22:49:36.933Z)

Interesting findings emerging. Note the `Visit user:pw@bad.example/x` case returns `[]` ? the `user:pw@bad.example` schemeless form is missed because of the `@` in userinfo. Also `foo@bad.example` (email) is skipped (good ? avoids false positive on emails).

The most interesting one: `foo@bad.example` where the URL is a schemeless domain preceded by `foo@` gets skipped entirely. That's intentional (email avoidance). But consider a schemeless bare domain that IS a real link preceded by non-space word chars. Let me now focus on the notable potential evasions: the userinfo trick and whether a blocked domain hidden in userinfo evades, plus IP normalization forms.

Let me probe more precisely the false-negative surface and IP obfuscation.

#### 15. user (2026-07-06T22:49:49.314Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro3.py:22: SyntaxWarning: invalid escape sequence '\s'
    print("=== newline / whitespace inside greedy [^\s] ===")
=== IP obfuscation: decimal / octal / hex ===
http://3627734734/ -> None
http://0330.0250.0.1/ -> 0330.0250.0.1
http://0xd8.0x3a.0xd6.0xce/ -> 0xd8.0x3a.0xd6.0xce
http://127.0.0.1 -> 127.0.0.1
http://[::ffff:127.0.0.1] -> ::ffff:127.0.0.1

=== schemeless bare domain no path, various preceders ===
'check bad.example now' -> [('bad.example', 'bad.example')]
'bad.example' -> [('bad.example', 'bad.example')]
'...bad.example' -> []

=== does blocked 'bad.example' match a URL whose host is user-controlled subdomain that ISNT really under it? ===
bad.example.evil.com matches bad.example?: False

=== newline / whitespace inside greedy [^\s] ===
'https://good.example/a\nhttps://bad.example/b' -> [('https://good.example/a', 'good.example'), ('https://bad.example/b', 'bad.example')]

=== extremely long text NOT truncated (caption stage caps 4096, url stage does not) ===
len 6021 found 1 in 0.0004 s
```

#### 16. assistant (2026-07-06T22:49:54.752Z)

These are all v1 metadata-blocklist limitations (IP obfuscation evasions are expected for a substring/exact blocklist, consistent with the ADR framing). The subdomain match is correct.

Now the most important check for a false-POSITIVE (worse category). Let me examine the `\_URL\_RE` for the schemeless branch matching things that aren't links ? like version numbers, filenames, sentence fragments ? which could wrongly extract a domain. And critically: does the schemeless branch's greedy path allow the userinfo `bad.example@evil.example` case to block on `evil.example` only (already saw domain is evil.example). Let me test false-positive triggers.

#### 17. user (2026-07-06T22:50:02.436Z)

```
=== false-positive candidates: non-link text that looks domain-ish ===
'The file is report.final.pdf' -> [('report.final.pdf', 'report.final.pdf')]
'version 2.0.zip download' -> [('2.0.zip', '2.0.zip')]
'Rated 9.5.io on scale' -> [('9.5.io', '9.5.io')]
'See page 3.co example' -> [('3.co', '3.co')]
'run npm.install now' -> [('npm.install', 'npm.install')]
'config.yaml settings' -> [('config.yaml', 'config.yaml')]
'a.mp4 and b.mov' -> [('a.mp', 'a.mp'), ('b.mov', 'b.mov')]
```

#### 18. assistant (2026-07-06T22:50:09.146Z)

This confirms false-positive extraction of non-links (filenames, version strings). But this only becomes a real false-positive **block** if the extracted "domain" matches a blocked entry. Since `.pdf`/`.zip`/`.yaml` aren't real TLDs an operator would block, the practical false-positive

risk is low ? the extraction is noisy but the blocklist gate limits harm. This is `link\_count` metadata inflation and a v1 noise issue, not a delivery-breaking bug on its own. Worth a low note.

Now let me verify the actual end-to-end short-circuit independently and check `content\_item.link\_url` existence. Let me check the ContentItem model.

#### 19. user (2026-07-06T22:50:10.700Z)

```
26: - ``LINK``: send a text message with a t.me link + metadata summary.
149:class ContentItem(BaseModel):
153:     (`content_type`='video'); `text` / `link_url` are landed now but stay
169:     text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170:     link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
397:     text: str | None = Field(default=None, description="Caption / message text")
```

#### 20. user (2026-07-06T22:50:11.598Z)

```
src/tg_video_filter/db.py:117:     link_url          TEXT,
src/tg_video_filter/db.py:870:         link_url=row["link_url"],
src/tg_video_filter/db.py:1183:         text, link_url, media_ref, document_id, duration_s,
width, height,
src/tg_video_filter/db.py:1193:         item.link_url,
src/tg_video_filter/models.py:153:     (`content_type`='video'); `text` / `link_url` are
landed now but stay
src/tg_video_filter/models.py:170:     link_url: str | None = Field(default=None,
description="For link/article posts (Phase 1: NULL)")
src/tg_video_filter/policy.py:435:         urls = extract_caption_urls(ctx.content_item.link_url)
+ extract_caption_urls(
Binary file src/tg_video_filter/__pycache__/db.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/models.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/policy.cpython-313.pyc matches
```

#### 21. assistant (2026-07-06T22:50:14.966Z)

Confirmed: `link\_url` is a real model field (Phase 1: always NULL, like `text` was pre-F2). Reading it is harmless (empty scan returns `()`), a latent-but-correct dependency. Not a defect.

Now let me verify the end-to-end short-circuit independently and confirm no network I/O. Let me also run the full test suite to confirm the gate.

#### 22. assistant (2026-07-06T22:51:27.621Z)

API Error: Connection closed mid-response. The response above may be incomplete.